

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA15 16
---------------------	---	---------------------	-------------

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Assessment Tool Weightage: 10%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I R.Jaideep(192525441)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: R.Jaideep

Assessment Tasks

Reverse Engineering

Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.
2. A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.
3. A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable big data security, streaming, and alerting mechanisms used.
4. A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.
5. A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference

Understanding of Architecture	25%	Shows clear understanding of	Good understanding	Partial understanding	Poor understanding
-------------------------------	-----	------------------------------	--------------------	-----------------------	--------------------

		underlying big data architecture			
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1.Reverse Engineering a Recommendation Engine Data Pipeline

A recommendation engine is designed to suggest relevant products, services, movies, or content to users based on their previous activities. In a modern system, user interactions such as **clicks, search queries, and purchases** are collected from multiple devices, including smartphones, tablets, laptops, and smart applications. By reverse engineering the system, the probable data types, ingestion flow, storage process, and analytics stages can be identified.

1. Probable Data Types

The first stage is to identify the types of data generated by users.

Clickstream data is generated whenever a user clicks a product, opens a page, watches content, or adds an item to a cart. It may contain user ID, product ID, timestamp, device ID, session ID, page URL, and click type. This is generally **semi-structured event data**, often represented in JSON format.

Search query data contains the words or phrases entered by users into the search system. It may include user ID, search text, timestamp, device information, location, and selected results. Search data can be treated as semi-structured data, while the actual query text is unstructured text.

Purchase data is more structured. It can include customer ID, order ID, product ID, quantity, price, payment status, and purchase timestamp. This information is commonly stored in relational databases.

The system also receives **user and device information**, such as age group, preferences, device type, operating system, application version, and session details. Product information such as product category, price, brand, and description may also be required by the recommendation engine.

2. Data Ingestion Flow

The ingestion process begins when users interact with the application through different devices. Each device generates events such as clicks, searches, product views, and purchases.

A **data collection layer** captures these events using APIs, application logs, or software development kits (SDKs). Since millions of events may be generated continuously, a message-broker platform such as **Apache Kafka** can be used to collect and transport the events.

The ingestion architecture can be divided into two paths. **Real-time data** such as clicks and searches can be streamed through Kafka for immediate processing. **Historical purchase and customer data** can be transferred through batch-based ETL/ELT pipelines.

Before storage, the data may be validated and standardized. Events from different devices should be associated with a common customer or session identifier. This helps the system build a unified view of a user's behavior across multiple devices.

3. Data Storage

The collected data can be stored in a **cloud data lake or distributed storage system**. Raw event data can be retained for future analysis, while cleaned and structured information can be stored in analytical databases or data warehouses.

A NoSQL database may be used for high-speed access to user profiles and product information. Metadata such as timestamps, device type, product category, and event type should be maintained so that the data can be efficiently searched and processed.

4. Data Processing and Feature Engineering

The next stage involves cleaning and transforming the collected data. Duplicate events, invalid records, missing values, and inconsistent formats should be identified and corrected.

A distributed processing framework such as **Apache Spark** can process large volumes of historical and streaming data. Important features can then be created from user behavior. For example, the system can calculate the number of product views, search frequency, purchase frequency, recently viewed products, average spending, and preferred categories.

User behavior can also be represented through **user-item interaction matrices**, where rows represent users and columns represent products or content. The interaction strength can be calculated based on clicks, searches, cart additions, and purchases.

5. Analytics and Recommendation Stage

The analytics layer uses machine-learning algorithms to generate recommendations. **Collaborative filtering** can recommend products based on the behavior of similar users. For example, if users who purchased Product A frequently purchased Product B, Product B can be recommended to other users who purchase Product A.

Content-based filtering can recommend products with characteristics similar to products a user has previously viewed or purchased. A hybrid recommendation system can combine collaborative filtering, content-based techniques, search behavior, and real-time interactions.

Machine-learning models can be trained using historical interaction data. The system can evaluate recommendations using metrics such as **precision, recall, click-through rate, conversion rate, and recommendation accuracy**.

6. Real-Time Recommendation Flow

For real-time personalization, recent user activity can be processed immediately. For example, if a user searches for a laptop and clicks several laptop products, the system can update the user's short-term interests and immediately recommend related accessories or similar laptops.

The final recommendations are delivered through the website or mobile application using an API. The user's response to those recommendations is again captured as new clickstream data, creating a **continuous feedback loop** that improves future recommendations.

2.Reverse Engineering the Distributed Backbone of a Ride-Sharing System

A ride-sharing company that provides **live traffic heatmaps and demand forecasts** must process large volumes of data generated continuously from GPS-enabled vehicles, mobile applications, traffic sensors, maps, and ride transactions. Since this information changes rapidly, the company requires a distributed storage and processing architecture capable of handling both **real-time streaming data and historical data**. By reverse engineering the visible features of the application, the likely backend can be identified as a combination of distributed data ingestion, scalable storage, stream processing, batch analytics, and machine-learning systems.

1. Data Sources

The system first receives data from multiple sources. Driver and passenger mobile applications continuously generate **GPS coordinates, speed, direction, location, trip status, and timestamps**. Ride requests provide information about pickup locations, destinations, request times, and ride cancellations. Additional data may come from traffic sensors, mapping services, weather systems, and historical trip databases.

GPS and application events are usually semi-structured records, while transaction and trip information may be stored as structured data. The system must combine all these sources to understand current traffic conditions and predict future demand.

2. Distributed Data Ingestion

Because thousands or millions of location events can be generated simultaneously, a distributed message-broker system such as **Apache Kafka** is a likely component of the ingestion layer. Each GPS update can be treated as an event and published to Kafka topics.

Kafka provides high-throughput ingestion and allows multiple processing services to consume the same data independently. For example, one service can process traffic conditions, another can update driver locations, and another can analyze demand patterns.

Data from external systems can be integrated through APIs and batch-based ETL/ELT pipelines. This creates a unified stream of information for downstream processing.

3. Distributed Storage Backbone

The company would require different storage systems for different workloads. A **cloud data lake or distributed object storage** can store large volumes of historical GPS traces, trip records, traffic information, and other raw data.

A distributed database or NoSQL system can be used for frequently changing information such as **current driver locations, active rides, and demand levels**. Data can be partitioned geographically or by time so that information for a particular city, zone, or time period can be accessed quickly.

Replication across multiple nodes provides fault tolerance. If one storage node fails, another copy of the data remains available. This is essential for a ride-sharing platform that operates continuously.

4. Real-Time Processing

Live traffic heatmaps require low-latency processing. A framework such as **Apache Spark Streaming or Apache Flink** can consume GPS events from Kafka and process them in real time.

The processing engine can group vehicle locations into geographic zones or map segments and calculate average speed, vehicle density, travel time, and congestion levels. The results can then be stored in a fast database or cache and delivered to the application.

For example, if many vehicles in a particular road segment suddenly slow down, the system can identify the segment as congested and update the traffic heatmap within seconds.

5. Batch Processing and Demand Forecasting

Demand forecasting requires historical information in addition to real-time data. Historical ride requests, completed trips, cancellations, time of day, day of week, weather conditions, holidays, and previous demand patterns can be processed using distributed batch frameworks such as **Apache Spark**.

Machine-learning models can use these features to predict the number of ride requests expected in different geographic zones. For example, the system may predict high demand near offices during morning hours or near entertainment areas during weekends.

The forecasts can help the company position drivers in high-demand areas and reduce passenger waiting time.

6. Caching and Application Layer

Since traffic maps and demand information must be delivered quickly to thousands of users, a **distributed caching layer** can store frequently requested results. Technologies such as Redis can provide fast access to current traffic and demand information.

An API layer can then deliver processed results to driver and passenger applications. This separates the user-facing applications from the underlying Big Data processing infrastructure.

3.Reverse Engineering a Big Data Fraud Detection System

A fraud detection dashboard that flags suspicious card transactions in near real time is likely supported by a **distributed Big Data architecture** combining secure data ingestion, real-time stream processing, machine-learning models, and automated alerting. Since financial transactions are generated continuously at high volume, the system must process data with very low latency while maintaining strong security and reliability.

1. Transaction Data Sources

The system receives transaction information from multiple sources such as **credit and debit card transactions, ATM withdrawals, online payments, mobile banking applications, merchant systems, and payment gateways**. Each transaction may contain card or customer identifiers, transaction amount, merchant information, location, timestamp, device details, and payment status.

Historical transaction records are also required because fraud detection models need previous legitimate and fraudulent transactions for training and comparison.

2. Secure Data Ingestion

The first stage is a high-speed ingestion layer. A distributed messaging platform such as **Apache Kafka** is a probable component because it can handle large numbers of transaction events continuously.

Each transaction can be published as an event to Kafka topics. Different services can consume these events simultaneously for fraud scoring, transaction monitoring, logging, and storage.

Security should be applied during ingestion using **TLS encryption, authentication, access controls, and secure API gateways**. Sensitive card information should be tokenized or masked so that unnecessary systems do not receive complete card details.

3. Distributed Storage

The system requires both real-time and historical storage. A **cloud data lake or distributed object-storage system** can store large volumes of historical transaction data, while a distributed database or data warehouse can provide fast analytical queries.

Data can be partitioned by date, geographic region, transaction type, or customer segment. Replication provides fault tolerance and ensures that transaction information is not lost if a storage node fails.

4. Real-Time Stream Processing

After ingestion, transactions are passed to a real-time processing engine such as **Apache Spark Streaming** or **Apache Flink**. The engine can examine each transaction within milliseconds or seconds.

The system can calculate features such as transaction frequency, transaction amount, distance from previous transactions, unusual merchant locations, device changes, and spending patterns. For example, if a card is used in two distant locations within a very short period, the transaction may receive a high fraud score.

5. Fraud Detection and Machine Learning

The core analytics layer can combine **rule-based detection and machine-learning models**. Simple rules can identify known suspicious conditions, such as unusually large transactions or repeated failed attempts.

Machine-learning models such as **Logistic Regression, Random Forest, Gradient Boosting, or anomaly-detection algorithms** can calculate the probability that a transaction is fraudulent. The model can consider many features simultaneously rather than depending on a single rule.

The output can be a **fraud risk score**, for example, low, medium, or high risk. High-risk transactions can be sent immediately to the alerting system.

6. Alerting and Dashboard Mechanism

When a transaction crosses a predefined risk threshold, the system generates an automated alert. Alerts can be sent to a fraud investigation dashboard, security team, or transaction-control service.

The dashboard can display information such as **transaction ID, amount, location, merchant, risk score, reason for detection, and transaction status**. Investigators can then review the transaction and decide whether it should be approved, blocked, or investigated further.

7. Security, Monitoring, and Audit

Since financial transactions are highly sensitive, the complete pipeline should include **encryption, authentication, authorization, tokenization, network security, and audit logging**. Every access to sensitive information should be recorded.

Continuous monitoring can detect unusual access patterns, failed authentication attempts, data transfers, and system failures. Security logs can be stored separately and protected from unauthorized modification. Regular audits can help verify compliance with applicable financial data-protection requirements.

4.Reverse Engineering Structured and Unstructured Data Management in Social Media

A social media platform generates and stores enormous amounts of **videos, text comments, hashtags, user profiles, likes, shares, views, and engagement metrics** every day. These data types have different structures and storage requirements. By reverse engineering the platform, it is possible to identify how structured and unstructured data are probably collected, stored, processed, and integrated to provide fast and personalized services.

1. Identification of Data Types

The first step is to classify the data generated by users. **User profiles, account details, follower relationships, and engagement counts** such as likes, shares, and views are mainly structured data. These records contain clearly defined fields and can be stored in relational or distributed databases.

Text comments and hashtags are generally semi-structured or unstructured data. A comment may contain text, user ID, post ID, timestamp, and language information. Hashtags can be stored as text but can also be indexed separately for fast searching and trending analysis.

Videos and images are unstructured data because they do not follow a fixed table format. They are also large in size and require scalable storage. Therefore, the platform would probably use distributed object storage or a cloud-based data lake for media files.

2. Data Ingestion

When a user uploads a video, writes a comment, adds a hashtag, or interacts with a post, the application generates an event. An ingestion layer receives these events through APIs and application services.

A distributed messaging system such as **Apache Kafka** can be used to transport high-volume events. For example, a video-upload event can contain metadata such as user ID, video ID, timestamp, and file location. Engagement events such as likes and shares can also be streamed through Kafka.

The actual video file would normally be transferred directly to distributed object storage, while its metadata is stored separately. This prevents large media files from unnecessarily passing through transactional databases.

3. Storage of Unstructured Data

Videos and images require large-scale and highly available storage. A **cloud object-storage system or distributed data lake** can store these media files. Files can be divided logically according to date, region, content type, or other metadata.

Multiple copies or replicas can provide fault tolerance. A Content Delivery Network (CDN) can also cache popular videos closer to users, reducing latency and improving playback performance.

The database does not need to store the complete video. Instead, it can store metadata such as **video ID, creator ID, file location, upload time, duration, format, and privacy status**. The application uses this information to retrieve the actual media file from object storage.

4. Storage of Structured Data

Structured information such as user accounts, relationships, engagement counts, and transaction records can be stored in **relational databases, distributed SQL databases, or NoSQL databases**, depending on the workload.

For example, a record may contain user ID, post ID, number of likes, number of comments, and timestamp. Distributed databases can replicate this information across multiple servers to provide high availability and support millions of simultaneous users.

Graph databases or graph-oriented data models can also be useful for representing relationships between users, followers, posts, and interactions.

5. Integration of Structured and Unstructured Data

The important part of the architecture is connecting the metadata with the actual unstructured content. A unique **video ID or post ID** can act as a common reference.

For example, a video file may be stored in object storage, while its metadata is stored in a database. When users like or comment on the video, those events reference the same video ID. This creates a connection between the large unstructured media file and structured engagement information.

A data lake can provide a unified environment for combining raw videos, comments, hashtags, logs, and structured records for large-scale analytics.

6. Processing and Analytics

Distributed processing frameworks such as **Apache Spark** can process both structured and unstructured datasets. Text-processing algorithms can analyze comments and hashtags to identify sentiment, topics, abusive content, and trending subjects.

Video-processing systems can generate metadata such as duration, resolution, objects, scenes, or speech transcripts. Machine-learning models can then use this information to support content recommendation and moderation.

7. Security and Data Management

Because the platform contains personal information and user-generated content, security controls are required. **Encryption, authentication, authorization, access control, and audit logging** should protect both structured and unstructured data.

Data-retention policies can determine how long inactive content and logs should be stored. Backup and replication mechanisms protect important data against hardware failures or accidental deletion.

5.Reverse Engineering a Manufacturing IoT Data Architecture

A manufacturing IoT platform collects huge amounts of data from machines, sensors, production systems, and maintenance applications across multiple plants. The data may include **machine logs, temperature readings, vibration measurements, equipment status, production rates, error codes, and maintenance events**. By reverse engineering the platform, the likely integration tools, storage strategy, processing systems, and analytics path can be identified.

1. Data Sources and Collection

The first stage consists of collecting data from machines installed across different manufacturing plants. Sensors attached to machines can continuously generate information about **temperature, pressure, vibration, energy consumption, speed, and machine status**. Machine controllers and industrial systems can also generate operational logs containing error codes, warnings, and production information.

Maintenance applications generate additional structured information such as machine ID, maintenance date, technician details, replaced components, repair duration, and maintenance type. Therefore, the platform must integrate both continuously generated IoT data and business-oriented maintenance records.

2. Probable Integration and Ingestion Tools

An IoT platform would likely use **IoT gateways** at each plant to collect data from local machines and sensors. These gateways can convert industrial protocols into standard formats and perform initial filtering before sending information to the central platform.

For high-volume streaming data, **Apache Kafka** or a similar message-broker system can be used. Kafka can receive machine events continuously and distribute them to multiple downstream applications. For example, one consumer can perform real-time monitoring while another stores the data for historical analysis.

Tools such as **Apache NiFi** can also be used for data routing, transformation, and integration. APIs and ETL/ELT pipelines can connect maintenance databases, ERP systems, production systems, and cloud applications.

3. Storage Strategy

The platform requires different storage systems because machine data has different formats and workloads. A **cloud data lake or distributed object storage** can store raw machine logs, sensor streams, and historical records. This provides scalable storage as the number of plants and connected machines increases.

Structured maintenance information can be stored in a **relational database or cloud data warehouse**. A time-series database can be used for sensor measurements because temperature, vibration, and pressure values are strongly associated with timestamps.

Data can be partitioned by **plant, machine, sensor type, and time period** to improve query performance. Replication and backups should be implemented to protect information from hardware failures and data loss.

4. Data Processing Path

After ingestion, the data passes through a processing layer. Real-time data can be processed using **Apache Spark Streaming or Apache Flink**. The processing system can clean sensor readings, remove duplicate events, handle missing values, and detect abnormal measurements.

For example, if a machine's vibration level suddenly increases above its normal operating range, the streaming system can identify the event immediately and generate an alert.

Historical data can be processed using distributed batch processing with **Apache Spark**. Data from different plants can be combined to identify long-term equipment behavior and production patterns.

5. Analytics and Machine Learning

The analytics stage converts machine data into useful operational insights. **Descriptive analytics** can show machine utilization, downtime, production rates, maintenance frequency, and energy consumption.

Predictive analytics can use machine-learning algorithms to predict equipment failures before they occur. Features such as vibration level, temperature, operating hours, previous failures, error codes, and maintenance history can be used to train prediction models.

For example, if historical data shows that a particular vibration pattern usually occurs before motor failure, the model can identify the same pattern in real-time sensor data and predict a possible failure.

6. Integration with Maintenance Systems

The analytics results should be integrated with the company's **Computerized Maintenance Management System (CMMS)** or enterprise resource planning system. When the analytics system predicts a possible equipment failure, it can automatically generate a maintenance alert or work order.

Dashboards can display the health status of machines across all plants. Managers can compare plants, identify frequently failing equipment, monitor downtime, and plan maintenance activities.